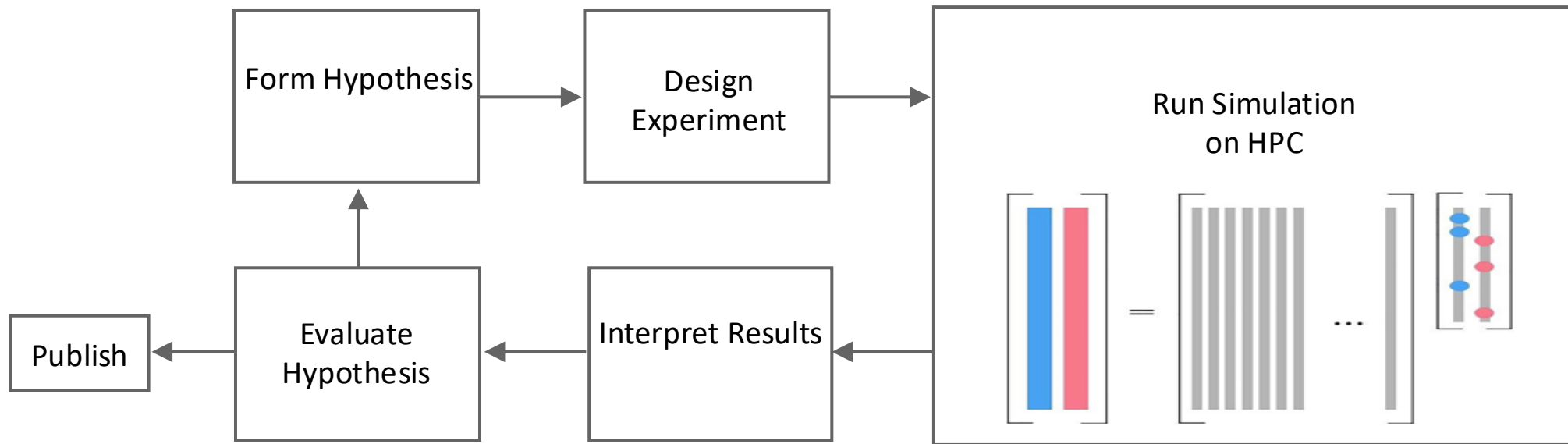


AI Agents for Scientific Workflows

Luke Smith
Research Associate
Texas Advanced Computing Center
July 27, 2026

Scientific Workflows Defined

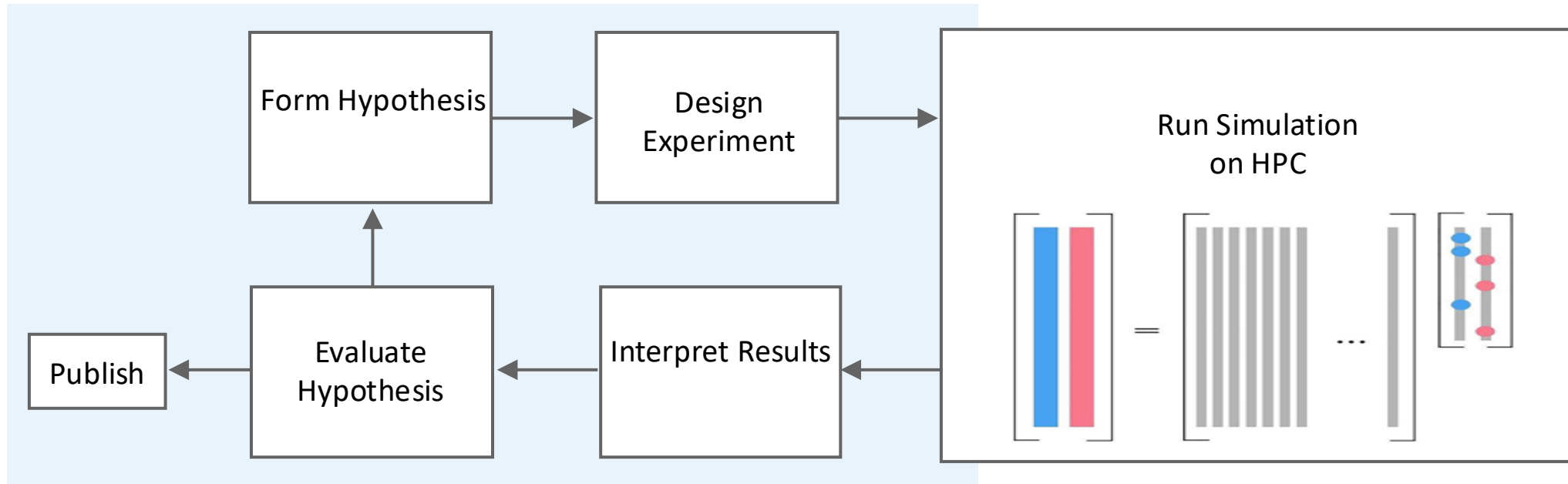
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**.
Computation can be numerical simulation, data processing, or visualization.



A scientific workflow is typically represented as a **Directed Acyclic Graph (DAG)**, in which nodes represent tools/actions and edges represent the flow of data.

Scientific Workflows Defined

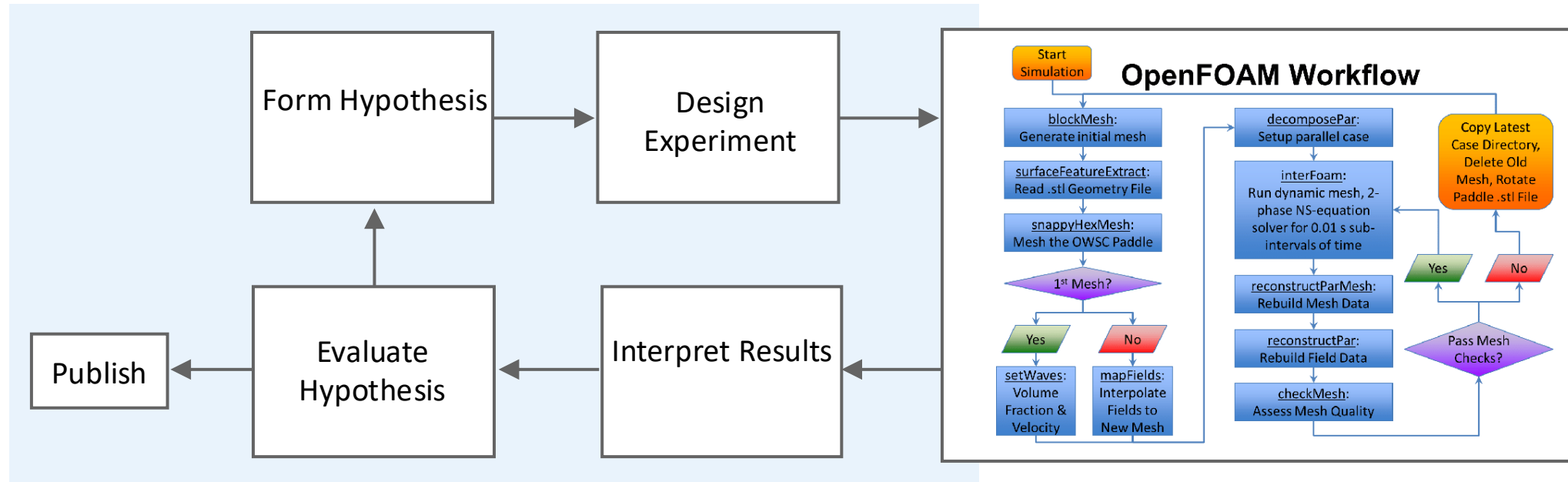
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**.
Computation can be numerical simulation, data processing, or visualization.



The bulk of scientific reasoning occurs on the left-hand side of this diagram, which represents the process of refining a hypothesis based on observation.

Scientific Workflows Defined

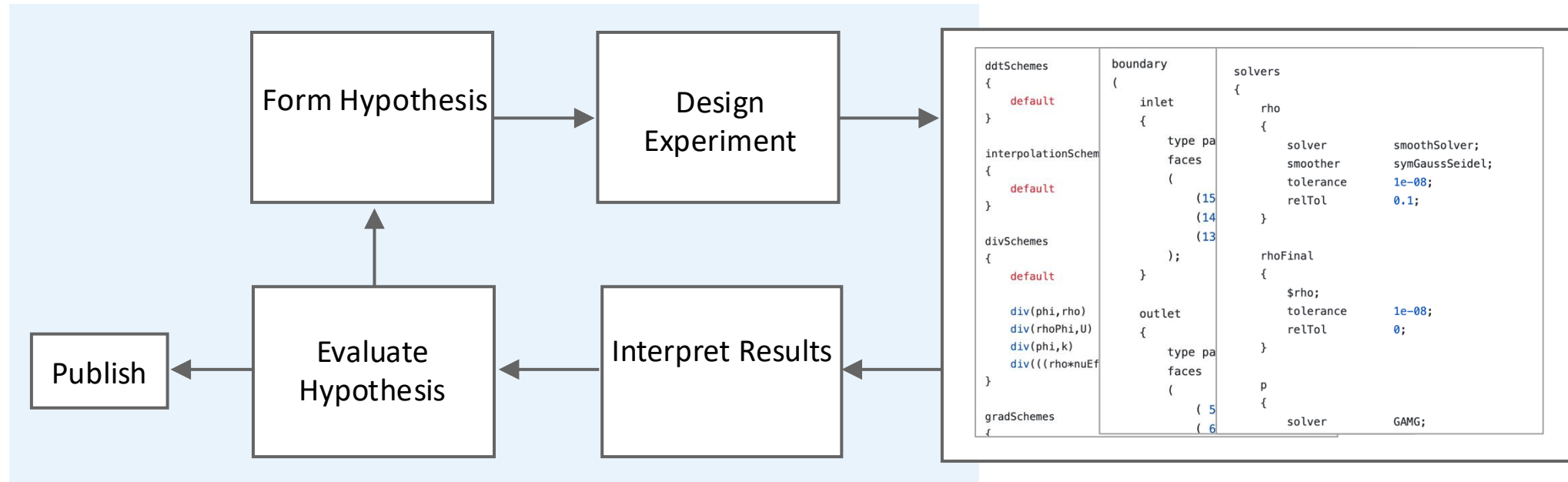
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**.
Computation can be numerical simulation, data processing, or visualization.



However, with growing software stacks and complex legacy code bases, computational scientist often spend undue time on the right-hand side of this diagram. It can require several years for a new student to become familiar with just one or two of these workflows.

Scientific Workflows Defined

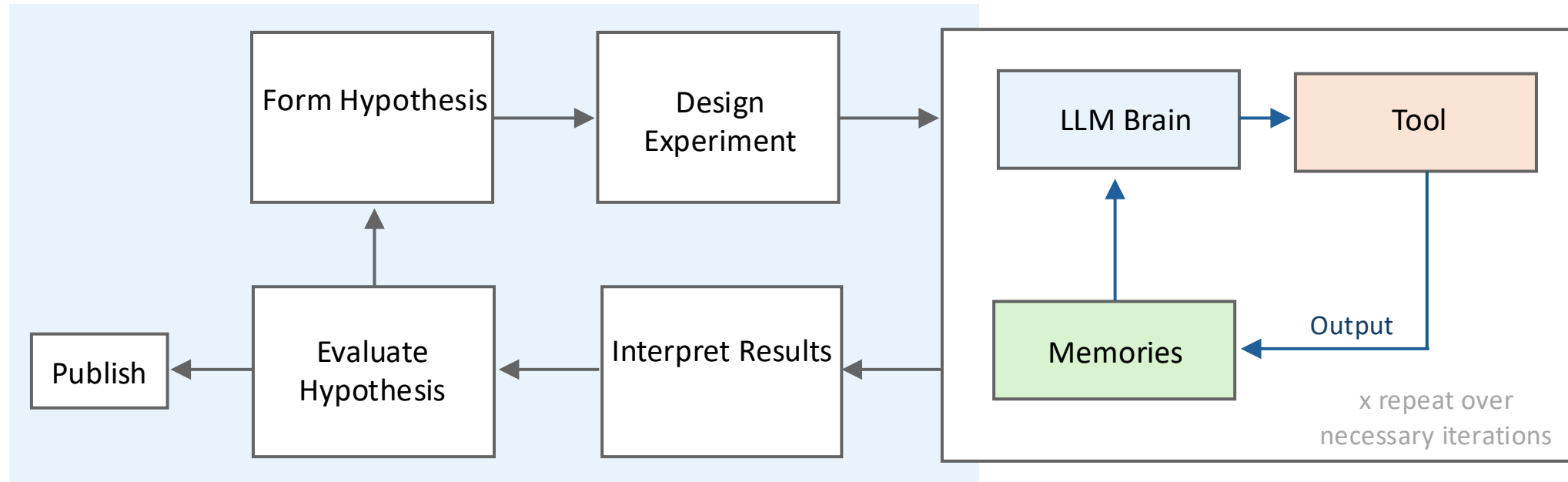
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**.
Computation can be numerical simulation, data processing, or visualization.



However, with growing software stacks and complex legacy code bases, computational scientist often spend undue time on the right-hand side of this diagram. It can require several years for a new student to become familiar with just one or two of these workflows.

How can AI agents help?

AI agents enable us to automate the setup and execution of a simulation or experiment, allowing the user to focus refining their hypothesis and designing new experiments.

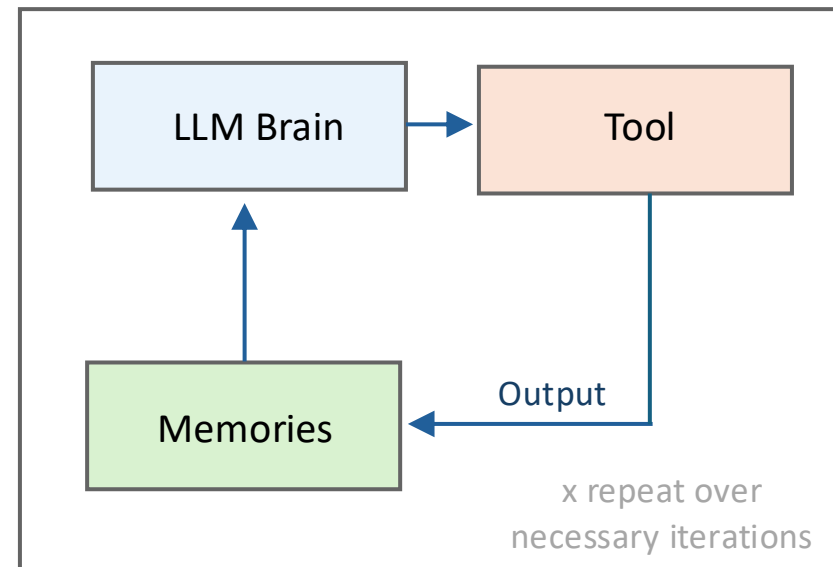


How can AI agents help?

AI agents enable us to automate the setup and execution of a simulation or experiment, allowing the user to focus refining their hypothesis and designing new experiments.

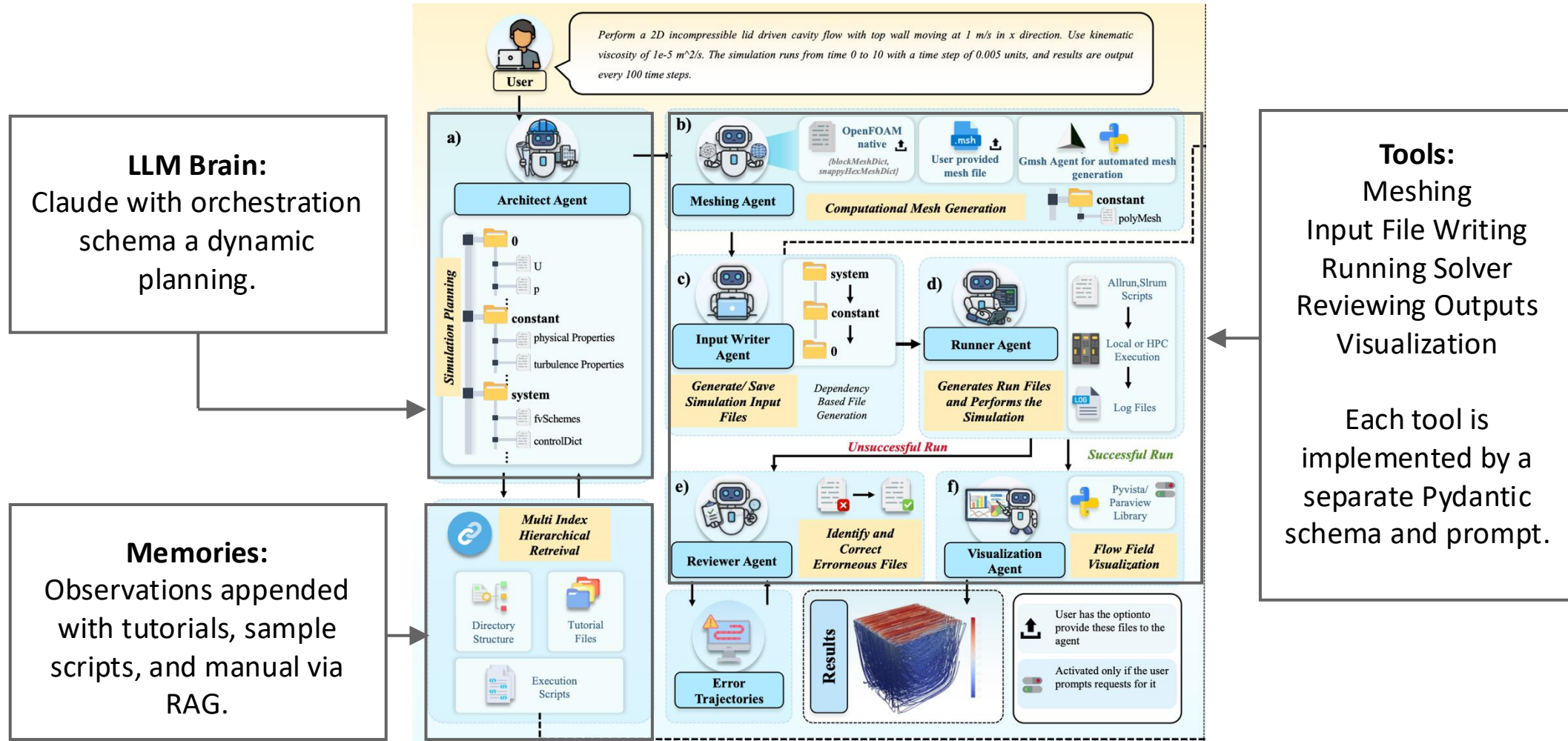
In the context of scientific workflows:

- **LLM Brain:**
Backend neural network with reasoning module (multistep planning or ReAct)
- **Tool:**
Physics simulator, data processing scheme
- **Memories:**
Agent prompt, previous observations, domain-specific context (user manuals, style-guides, etc.)



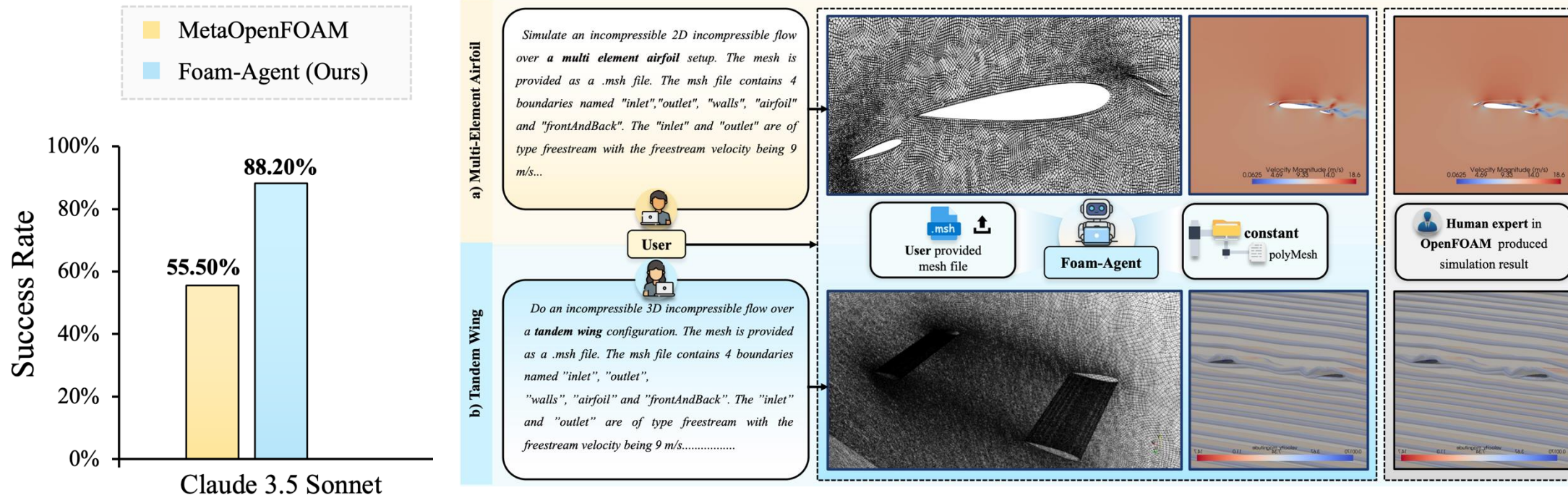
We automate a given scientific workflow by mapping to the three core components of an agentic system: reasoning, tools, and memories.

Example 1: Foam-Agent for Computational Fluid Dynamics



Source: Yu et al, "Foam Agent: Toward Automated Intelligent CFD Workflows" (2026)

Example 1: Foam-Agent for Computational Fluid Dynamics



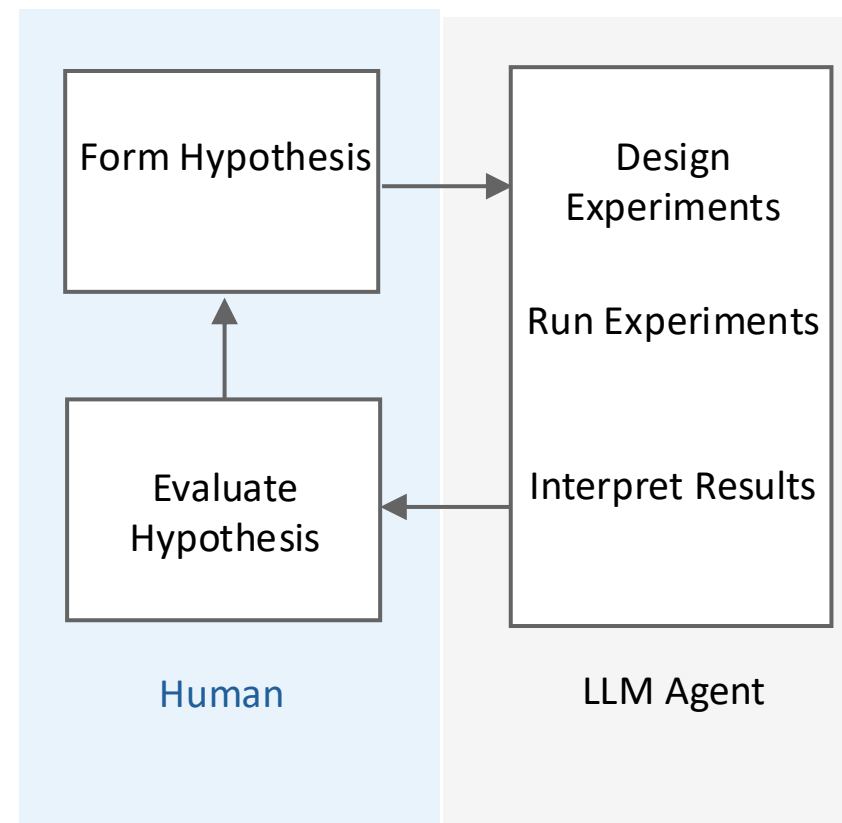
Source: Yu et al, "Foam Agent: Toward Automated Intelligent CFD Workflows" (2026)

- The author's report a success rate of about 90% on a benchmark of canonical steady flows.
- Success was defined based on the presence of a stable output flowfield. Benchmark metrics based on the **fidelity** of the flowfield were not reported.
- Still, those who have run OpenFOAM know that arriving at a stable solve is one of the more difficult steps of the workflow. Fidelity is typically achieved by fine-tuning a base configuration like this.

Example 2: Self-Driving Labs

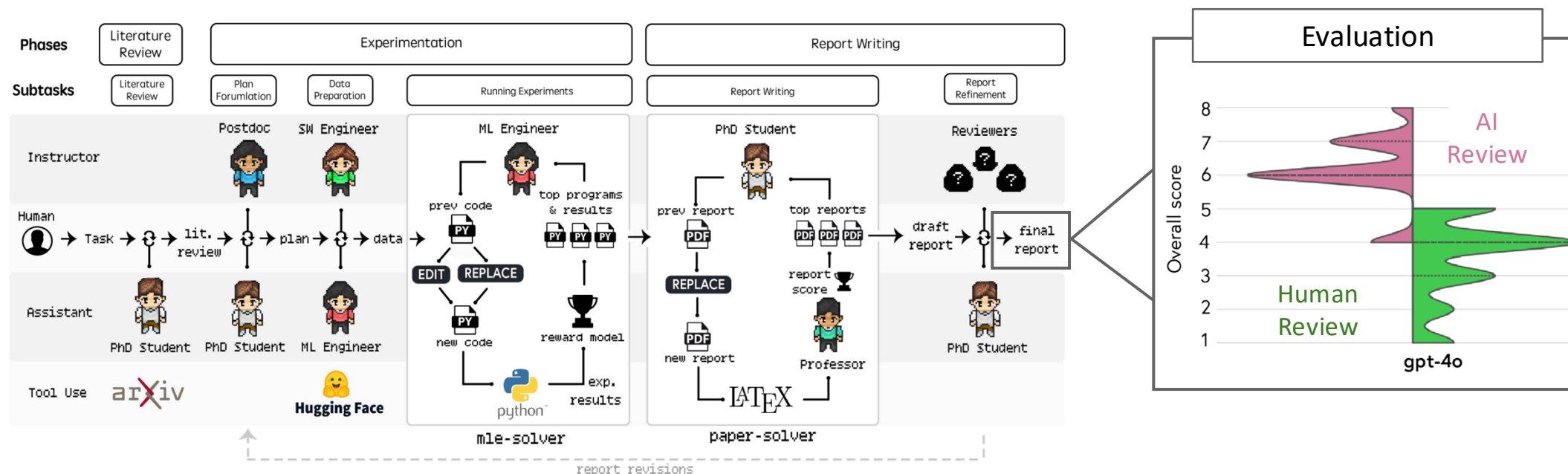


Source: Vertical Cloud Lab at BYU (<https://github.com/vertical-cloud-lab>)



- More aggressive workflows permit the agent to make decisions about which experiments to run – ideal when one expects a large parameter space with a small subset of interesting experiments.
- Note that agent automation is not limited to numerical simulations! Any system with a software interface can be designated as an agent's tool call.

Example 3: Agent Laboratory



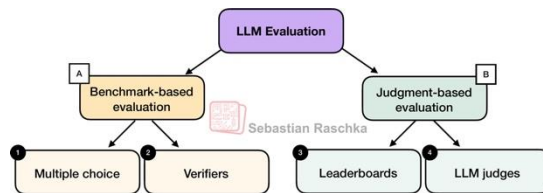
Source: Schmidgall et al, "Agent Laboratory: Using LLM Agents as Research Assistants" (2025)

- More extreme workflows attempt to automate the **entire end-to-end research process**, including formulating a hypothesis, performing a literature review, running experiments, and writing a paper.
- The problem with these workflows is analogous to error accumulation: the more agents we incorporate, the more bias we ultimately introduce. In this case, that manifests in poor-quality papers.

Limitations

The development of AI agents for software engineering has been rapid and eventful, but there are still several open questions that limit their widespread adoption in scientific computing.

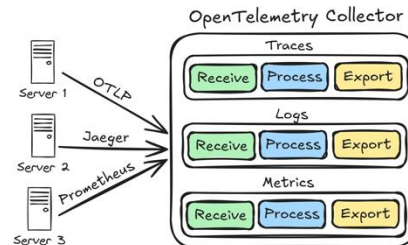
Evaluation



Problem: There are no universal metrics for assessing the accuracy of an agent's outputs, particularly for text-only outputs.

Potential Solutions: Functional scorers, LLM-as-judge, development of benchmarks with a quantitative ground truth.

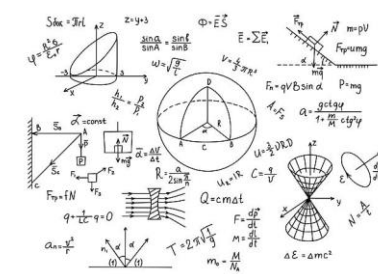
Tracing



Problem: LLM backends provide no means of monitoring their systems activity.

Potential Solutions: Tracing as a simultaneous process (Arize Phoenix), standardized guardrails that restrict what tool calls are available.

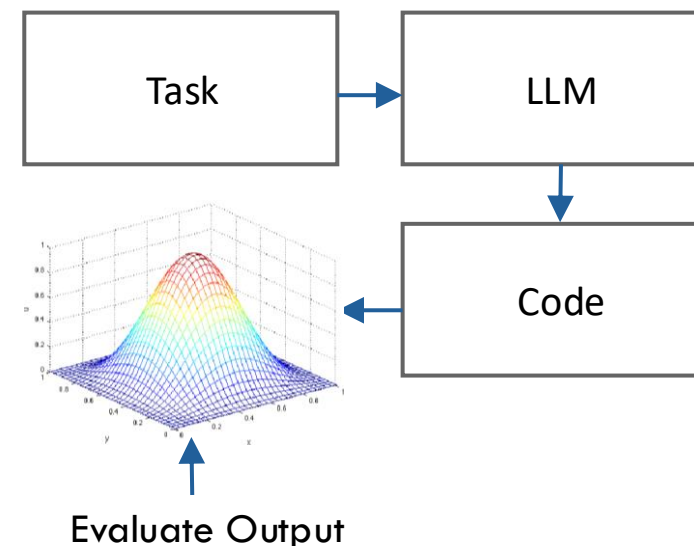
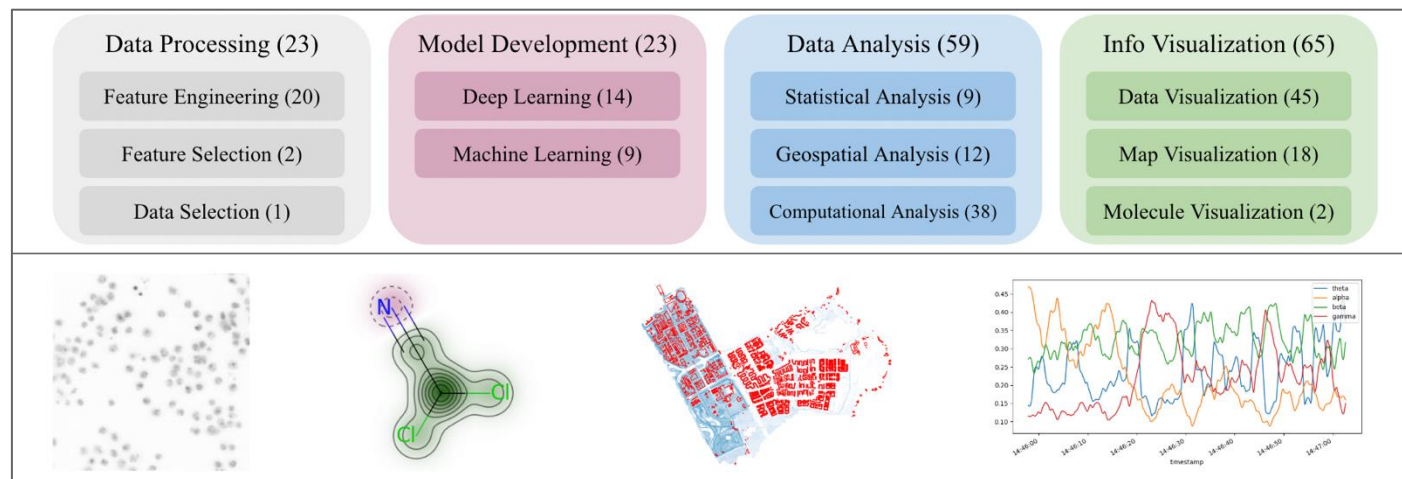
Domain Expertise



Problem: LLM's are generalist models, and scientific computing applications are highly specialized.

Potential Solutions: Custom schema for each piece of software (MPC), advanced reasoning and prompting, RAG, higher-capacity backends.

Relevant Literature: ScienceAgentBench



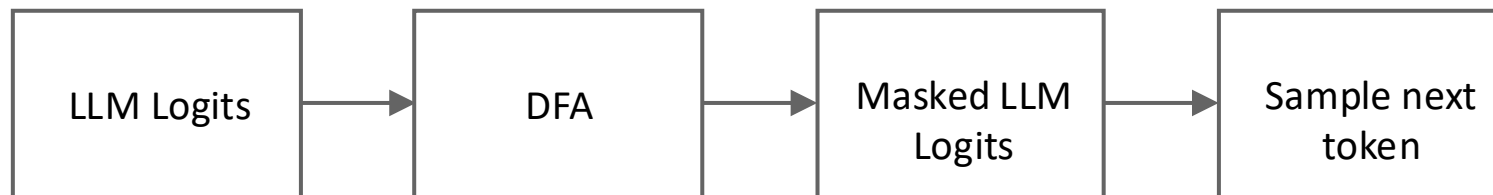
- Comprehensive benchmark that collects scientific computing tasks from several domains, including computational chemistry, information science, neuroscience, and bioinformatics. Each task has an expected, exact computational output and a reference script.
- Evaluation metrics: valid execution rate, success rate, CodeBERTScore, and LLM API cost, augmented with manual inspection from domain experts.
- Main Result: the best performing model (Claude 3.5-Sonnet) only succeeded on about 30% of benchmark tasks.

Demonstration

Our goal is to set up a tool call for an LLM to generate code. In most agent setups, tool calls are implemented via schema for structured generation. **Pydantic schema** look something like this:

```
class custom_schema(BaseModel):  
    thought: str = Field(..., description="What to do next")  
    property1: Optional[str] = Field(..., description = "")  
    property2: Optional[str] = Field(..., description = "")
```

The idea is that the LLM's logits are constrained such that it returns a response that adheres to the specified schema. The enforcement of schema can be implemented with the **outlines** library, which masks invalid logits based on a pre-computed map for each state in the schema:



Deterministic Finite Automaton:

Input: current state within schema

Output: list of valid tokens

Demonstration

In the demonstration that follows, we constrain the LLM to select an action from a finite set of tools. These tools include a calculator, a physics simulator, and a final summary. If the LLM selects the simulator, its outputs are returned to the LLM's context. If the LLM selects final summary, it exits the loop.

